

Justificación de transacciones y sus niveles de aislamiento

1. En el reporte del Administrador, en AdministratorReportRepository.cs

```
private string getAllCompaniesPayrollQuery()
{
    return @"
        BEGIN TRY
            BEGIN TRAN;
            WITH PagosLey AS (
                SELECT
                    emp.idPersonaFisica AS empleadorId,
                    pj.razonSocial AS empresaNombre,
                    pj.tipoPago AS frecuenciaPago,
                    dp.fechaInicio,
                    dp.fechaFin,
                    SUM(ROUND(dp.salarioBruto / 10.0, 0)) AS totalSalarios,
                    SUM(COALESCE(dap.monto, 0)) AS totalPagosLeyEmpleador
                FROM Empleador emp
                JOIN PersonaJuridica pj ON pj.id = emp.idPersonaJuridica
                JOIN Empleado e ON e.idEmpleadorContratador =
emp.idPersonaFisica
                JOIN PersonaFisica pfE ON pfE.id = emp.idPersonaFisica
                JOIN Contrato c ON c.idEmpleado = e.idPersonaFisica
                JOIN DetallePago dp ON dp.idEmpleado = e.idPersonaFisica
                LEFT JOIN DeduccionAPago dap
                    ON dap.idDetallePago = dp.id
                    AND dap.tipo LIKE 'empleador_%'
                WHERE dp.fechaInicio >= @startDate
                    AND dp.fechaFin <= @endDate
                GROUP BY
                    emp.idPersonaFisica,
                    pfE.primerNombre, pfE.segundoNombre,
                    pfE.primerApellido, pfE.segundoApellido,
                    pj.razonSocial, pj.tipoPago,
                    dp.fechaInicio, dp.fechaFin
            ),

            DeduccionesVol AS (
                SELECT
```

```

        emp.idPersonaFisica AS empleadorId,
        dp.fechaInicio,
        SUM(dap.monto) AS totalDeduccionesVoluntarias
    FROM DeduccionAPago dap
    JOIN DetallePago dp ON dp.id = dap.idDetallePago
    JOIN Empleado e ON e.idPersonaFisica = dp.idEmpleado
    JOIN Empleador emp ON emp.idPersonaFisica =
e.idEmpleadorContratador
    WHERE dap.tipo NOT LIKE 'empleador_%'
        AND dp.fechaInicio >= @startDate
        AND dp.fechaFin <= @endDate
    GROUP BY emp.idPersonaFisica, dp.fechaInicio
)

SELECT
    p.empresaNombre,
    p.frecuenciaPago,
    p.fechaInicio,
    p.fechaFin,
    p.totalSalarios,
    p.totalPagosLeyEmpleador,
    ISNULL(d.totalDeduccionesVoluntarias, 0) AS
totalDeduccionesVoluntarias
FROM PagosLey AS p
LEFT JOIN DeduccionesVol AS d
    ON d.empleadorId = p.empleadorId
    AND d.fechaInicio = p.fechaInicio
ORDER BY
    p.empresaNombre,
    p.fechaInicio DESC;

COMMIT TRAN;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRAN;
    THROW;
END CATCH;
";

```

Se utiliza una transacción explícita con manejo de errores para garantizar la consistencia de los datos. Este query realiza operaciones de lectura, se consulta información en múltiples tablas relacionadas lo que hace necesario asegurar que todos los datos sean coherentes entre sí en

el momento de ejecución. Por ello, se incluye una transacción que comienza al inicio del bloque TRY y se confirma con COMMIT si todo finaliza correctamente; en caso de error, el bloque CATCH realiza un ROLLBACK y relanza la excepción para que pueda manejarse correctamente a nivel de aplicación.

En cuanto al nivel de aislamiento, se utiliza el valor predeterminado de SQL Server, READ COMMITTED, el cual garantiza que las lecturas no verán datos de transacciones no confirmadas. Este nivel es adecuado para este escenario, ya que evita inconsistencias básicas sin incurrir en bloqueos prolongados, ofreciendo un equilibrio razonable entre integridad de los datos y rendimiento del sistema. De esta forma, se asegura que el reporte refleje un estado coherente de la base de datos sin interferencias de transacciones que aún no han finalizado.

2. Crear una nueva compañía, en CompanyRepository.cs

```
public bool createNewCompany(CompanyModel company)
{
    using (var connection = GetConnection())
    {
        connection.Open();
        using (var transaction = connection.BeginTransaction())
        {
            try
            {
                if (dataAlreadyExists("Persona", "identificacion",
                    , company.idNumber, transaction))
                    throw new Exception("CEDULA_DUPLICADA");

                if (dataAlreadyExists("Persona", "numeroTelefono",
                    , company.phoneNumber, transaction))
                    throw new Exception("TELEFONO_DUPLICADO");

                if (dataAlreadyExists("Persona", "correoElectronico",
                    , company.email, transaction))
                    throw new Exception("EMAIL_DUPLICADO");

                Guid naturalPersonId =
                    GetPersonIdByUsername(company.employerUsername);

                var auditId = insertAudit(company.employerUsername,
transaction);
                var personId = insertPerson(company, auditId, transaction);

                insertLegalPerson(company, personId, transaction);
            }
            catch { }
        }
    }
}
```

```

        insertAddress(company, personId, transaction);
        assignCompanyToEmployer(company, naturalPersonId, personId
            , transaction);

        transaction.Commit();
        Debug.WriteLine("Successful insertion in all tables.");
        return true;
    }
    catch (Exception ex)
    {
        Debug.WriteLine("Insert error: " + ex.Message);
        transaction.Rollback();
        throw;
    }
}
}
}

```

Se utiliza una transacción explícita para garantizar la integridad y consistencia de los datos durante el proceso de creación de una nueva empresa. Este proceso implica múltiples inserciones distribuidas en varias tablas relacionadas, como Persona, PersonaJuridica, Direccion y registros de auditoría, así como validaciones. Para lograr integridad de los datos, se inicia una transacción antes de realizar cualquier operación de base de datos, y solo se confirma con Commit si todo finaliza correctamente. En caso de que ocurra cualquier excepción, ya sea por datos duplicados o por errores en las inserciones, se realiza un Rollback que revierte todos los cambios realizados hasta ese punto, previniendo así inconsistencias o registros parciales en la base de datos.

En cuanto al nivel de aislamiento, se utiliza el nivel predeterminado de SQL Server, READ COMMITTED, el cual impide la lectura de datos no confirmados por otras transacciones. Este nivel es apropiado para este escenario ya que evita interferencias de otras operaciones concurrentes y asegura que las validaciones de duplicidad y las inserciones se basen en un estado estable y confiable de la base de datos.

3. En el reporte de Empleados, en EmployeeReportRepository.cs

```

public EmployeeFullReport GetEmployeePayrollReport(string employeeId)
{
    using var connection = connectionRepository.connection;
    connection.Open();

    using var transaction = connection.BeginTransaction();

```

```

    try
    {
        var rows = executeEmployeeQuery(connection, transaction,
employeeId);
        transaction.Commit();

        var result = buildEmployeeReport(rows);
        return result;
    }
    catch (Exception ex)
    {
        transaction.Rollback();
        throw new Exception("GetEmployeePayrollReport failed: " +
ex.Message
        , ex);
    }
}

```

Se utiliza una transacción explícita con el objetivo de asegurar que todos los datos recuperados del empleado sean consistentes entre sí y correspondan a un mismo periodo de pago. La transacción garantiza que si durante la ejecución ocurre algún cambio en los datos del empleado, como actualizaciones en su historial de pagos o información personal, dichos cambios no afecten los resultados del reporte que se está generando. Se abre una conexión a la base de datos y se inicia la transacción antes de ejecutar la consulta. Si la consulta se completa sin errores, la transacción se confirma con un Commit; de lo contrario, en caso de cualquier excepción, se realiza un Rollback para abortar el proceso y se relanza la excepción con un mensaje más descriptivo.

El nivel de aislamiento utilizado es el predeterminado de SQL Server, READ COMMITTED, el cual evita que se lean datos no confirmados por otras transacciones, brindando un mínimo nivel de protección frente a inconsistencias por escrituras concurrentes. Este nivel es adecuado para consultas como esta, donde se requiere coherencia sin introducir bloqueos excesivos. Así, se asegura que el reporte generado para un empleado refleje información íntegra y sin interferencias externas.

4. En actualizar la información de un empleado existe, en EmployeeRepository.cs

```

public void UpdateEmployeeData(EmployeeModel employee, Guid id, string
loggedUsername)
{
    using (var connection = GetConnection())

```

```
{

connection.Open();
using (var transaction = connection.BeginTransaction())
{
    try
    {
        var currentData = GetEmployeeCurrentData(id, transaction);
        if (currentData == null)
        {
            throw new Exception("EMPLOYEE_NOT_FOUND");
        }

        if (currentData.idNumber != employee.idNumber ||
            currentData.firstName != employee.firstName ||
            currentData.secondName != employee.secondName ||
            currentData.firstLastName != employee.firstLastName ||
            currentData.secondLastName != employee.secondLastName)
        {
            if (hasEmployeeBeenPayedAlready(id, transaction))
            {
                throw new Exception("EMPLOYEE_ALREADY_PAYED");
            }
        }

        if (employee.idNumber != currentData.idNumber) {
            if (dataAlreadyExists("Persona", "identificacion"
                , employee.idNumber, transaction)) {
                throw new Exception("CEDULA_DUPLICADA");
            }
        }

        if (employee.phoneNumber != currentData.phoneNumber) {
            if (dataAlreadyExists("Persona", "numeroTelefono"
                , employee.phoneNumber, transaction)) {
                throw new Exception("TELEFONO_DUPLICADO");
            }
        }

        if (employee.email != currentData.email) {
            if (dataAlreadyExists("Persona", "correoElectronico"
                , employee.email, transaction)) {
                throw new Exception("EMAIL_DUPLICADO");
            }
        }
    }
}
```

```

        if (employee.username != currentData.username) {
            if (dataAlreadyExists("Usuario", "nickname"
                , employee.username, transaction)) {
                throw new Exception("USERNAME_DUPLICADO");
            }
        }

        UpdatePerson(employee, id, transaction);

        UpdateNaturalPerson(employee, id, transaction);

        UpdateAddress(employee, id, transaction);

        UpdateUser(employee, id, transaction);

        UpdateEmployeeDetails(employee, id, transaction);

        UpdateEmployeeContractDetails(employee, id, transaction);

        UpdateEmployeeAudit(id, loggedUsername, transaction);

        transaction.Commit();
    }
    catch (Exception ex)
    {
        transaction.Rollback();
        throw new Exception("Error updating employee data: " +
ex.Message);
    }
}
}
}

```

Se hace uso de una transacción explícita para asegurar la integridad y consistencia de los datos durante el proceso de actualización de la información de un empleado. Este método realiza múltiples validaciones y actualizaciones sobre diferentes tablas de la base de datos, incluyendo datos personales, dirección, usuario, detalles laborales y contrato del empleado, así como el registro de auditoría correspondiente. Si alguna parte del proceso falla se realiza un Rollback de la transacción para revertir todos los cambios realizados hasta ese punto, evitando dejar datos en un estado parcial o inconsistente. Sólo si todas las validaciones y actualizaciones se completan correctamente, se confirma la transacción con un Commit.

En cuanto al nivel de aislamiento, se utiliza el predeterminado de SQL Server, READ COMMITTED, el cual impide la lectura de datos no confirmados por otras transacciones, garantizando que las validaciones de unicidad se realicen sobre un estado confiable de la base de datos. Este nivel resulta adecuado en este escenario, ya que proporciona un buen equilibrio entre integridad de los datos y eficiencia, especialmente al evitar interferencias causadas por transacciones concurrentes.

5. En el reporte de Empleador, en EmployeeReportRepository.cs

```
public EmployerPayrollReport GetEmployerPayrollReport(string employerId)
{
    using var connection = connectionRepository.connection;
    connection.Open();
    using var transaction = connection.BeginTransaction();
    try
    {
        var rows = executeEmployerQuery(connection, transaction,
employerId);
        var voluntaryDeductions =
executeVoluntaryDeductionsSumQuery(connection
    , transaction, employerId);
        transaction.Commit();

        foreach (var period in rows)
        {
            var periodoKey = (DateTime)period["fechaInicio"];
            if (voluntaryDeductions.TryGetValue(periodoKey
    , out decimal deducciones))
                period["total_deducciones_voluntarias"] = deducciones;
            else
                period["total_deducciones_voluntarias"] = 0m;
        }

        return buildEmployerReport(rows);
    }
    catch (Exception ex)
    {
        transaction.Rollback();
        throw new Exception("GetEmployerPayrollReport failed: " +
ex.Message
    , ex);
    }
}
```


Se utiliza una transacción explícita para garantizar la coherencia de los datos durante la generación del reporte de planillas de un empleador. Este proceso involucra la ejecución de múltiples consultas que extraen información relacionada con los pagos y deducciones voluntarias correspondientes a distintos periodos, datos que luego se combinan para construir el resultado final. Dado que estas consultas acceden a tablas que podrían estar siendo modificadas por otros procesos concurrentes, se utiliza una transacción para asegurar que toda la información se obtenga de forma consistente y sin interferencias durante su ejecución. La transacción se confirma con un Commit solo si ambas consultas se completan correctamente; de lo contrario, en caso de cualquier excepción, se realiza un Rollback para abortar el proceso y evitar devolver un reporte basado en datos parciales o incoherentes.

El nivel de aislamiento utilizado es el predeterminado en SQL Server, READ COMMITTED, que impide leer datos de transacciones no confirmadas, lo cual es suficiente para evitar inconsistencias básicas durante la ejecución de reportes. Esta configuración proporciona un equilibrio adecuado entre rendimiento y fiabilidad, asegurando que los datos leídos reflejen un estado estable y confirmado de la base de datos.

6. En obtener los resultados de planilla por empleador, en [PayrollOrchestratorRepository.cs](#)

```
public List<object> getPayrollsByEmployerId(string employerId)
{
    using var connection = connectionRepository.connection;
    connection.Open();

    using var transaction = connection.BeginTransaction();

    try
    {
        var dataBaseRows = executePayrollQuery(connection, transaction
            , employerId);

        transaction.Commit();

        return buildPayrollResults(dataBaseRows);
    }
    catch (Exception ex)
    {
        transaction.Rollback();
        throw new Exception("GetPayrollsByEmployerId failed: " + ex.Message,
ex);
    }
}
```

Se hace uso de una transacción explícita para asegurar la coherencia de los datos al consultar las planillas asociadas a un empleador específico. Aunque el método solo realiza operaciones de lectura, se utilizan múltiples tablas relacionadas que podrían estar siendo modificadas simultáneamente por otros procesos. Al envolver la consulta dentro de una transacción, se garantiza que los datos obtenidos correspondan a un estado consistente de la base de datos durante toda la ejecución. La transacción se confirma con un Commit únicamente si la consulta se ejecuta correctamente; en caso de cualquier error, se realiza un Rollback para evitar el uso de resultados incompletos o inconsistentes.

El nivel de aislamiento empleado es el predeterminado de SQL Server, READ COMMITTED, que previene la lectura de datos no confirmados por otras transacciones, ofreciendo así un grado básico de protección contra inconsistencias. Este nivel es apropiado para consultas como esta, donde se busca mantener la integridad de los datos leídos sin comprometer el rendimiento general del sistema.

7. Crear un nuevo empleado, en SQLQuery024.sql y en EmployeeRepository.cs

```
public bool createNewEmployee(EmployeeModel employee, string loggedId)
{
    using (var connection = GetConnection())
    {
        connection.Open();
        using (var transaction = connection.BeginTransaction())
        {
            try
            {
                var cmd = new SqlCommand("sp_CreateNewEmployee", connection
                    , transaction)
                {
                    CommandType = CommandType.StoredProcedure
                };

                addAuditParameters(cmd, loggedId);
                addPersonParameters(cmd, employee);
                addNaturalPersonParameters(cmd, employee);
                addAddressParameters(cmd, employee);
                addUserParameters(cmd, employee);
                addEmployeeParameters(cmd, employee, loggedId);
                addContractParameters(cmd, employee);

                cmd.ExecuteNonQuery();
                transaction.Commit();
                return true;
            }
            catch { }
        }
    }
}
```

```

    }
    catch (SqlException ex)
    {
        Debug.WriteLine("Stored procedure error: " + ex.Message);
        transaction.Rollback();
        throw;
    }
}
}
}

```

En el procedimiento almacenado `sp_CreateNewEmployee`, se implementa una transacción explícita para garantizar la atomicidad del proceso de creación de un nuevo empleado, el cual implica múltiples inserciones en distintas tablas (Persona, PersonaFisica, Direccion, Usuario, Empleado, Contrato, Auditoria) y validaciones previas sobre la unicidad de campos como número de identificación, teléfono, correo electrónico y nombre de usuario. La transacción se inicia al principio del bloque TRY y se confirma con un COMMIT solo si todo el proceso se realiza exitosamente. En caso de error, se ejecuta un ROLLBACK para revertir todos los cambios y se lanza nuevamente la excepción capturada, preservando así la integridad de los datos. También se realiza una validación del usuario que ejecuta la operación, extrayendo su nombre de usuario a partir del identificador recibido como parámetro.

El nivel de aislamiento utilizado es el predeterminado de SQL Server, READ COMMITTED, el cual previene la lectura de datos no confirmados y asegura que las validaciones de duplicidad se basen en un estado consistente de la base de datos. Este nivel es adecuado para operaciones de inserción como esta, ya que protege contra inconsistencias sin afectar negativamente el rendimiento.

8. Crear un nuevo empleador, en `SQLQuery029.sql` y en [EmployeeRepository.cs](#)

```

public bool createNewEmployer(EmployerModel employer)
{
    connection.Open();
    var isSuccess = false;

    try
    {
        using (var cmd = new SqlCommand("sp_CreateNewEmployer", connection))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            AddIdentificationInfo(cmd, employer);

```

```

        AddNameInfo(cmd, employer);
        AddAddressInfo(cmd, employer);
        AddUserInfo(cmd, employer);

        cmd.ExecuteNonQuery();
    }

    isSuccess = true;
}
catch (SqlException)
{
    throw;
}
finally
{
    connection.Close();
}

return isSuccess;
}

```

En el procedimiento almacenado `sp_CreateNewEmployer`, se utiliza una transacción explícita para asegurar que todo el proceso de creación de un nuevo empleador se realice de forma atómica y coherente. Este procedimiento implica múltiples operaciones como la verificación de unicidad de la cédula, teléfono, correo electrónico y nombre de usuario, así como la inserción de datos en varias tablas (Persona, PersonaFisica, Direccion, Usuario y Auditoria). Al envolver estas operaciones dentro de una transacción, se garantiza que todas ellas se completen exitosamente o, en caso de error, se deshagan por completo mediante un `ROLLBACK`, evitando así inconsistencias o registros incompletos. Además, el bloque `CATCH` captura cualquier excepción que ocurra durante la ejecución, permitiendo un manejo de errores robusto y una retroalimentación clara al usuario.

El procedimiento utiliza el nivel de aislamiento por defecto de SQL Server, `READ COMMITTED`, que impide la lectura de datos no confirmados por otras transacciones concurrentes. Este nivel es adecuado para este tipo de inserciones, ya que evita conflictos de concurrencia básicos sin introducir bloqueos innecesarios.

9. En crear un nuevo beneficio para la empresa, en `SQLQuery035.sql` y `CompanyBenefitRepository.cs`

```

public bool CreateBenefit(CompanyBenefitDTO companyBenefit, string
loggedUserNickname)

```

```

{
    try
    {
        if (_connection.State != ConnectionState.Open)
            _connection.Open();

        using (var transaction = _connection.BeginTransaction())
        {
            companyBenefit.benefit.companyId =
getCompanyId(loggedUserNickname, transaction);
            if (!companyBenefit.benefit.companyId.HasValue)
                throw new Exception("EMPRESA_NO_ENCONTRADA");

            if (dataAlreadyExists("Beneficio", "nombre",
companyBenefit.benefit.name, companyBenefit.benefit.companyId,
transaction))

                throw new Exception("BENEFICIO_DUPLICADO");
            using (var cmd = new SqlCommand("StoreBenefit",
_connection, transaction))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@User",
loggedUserNickname);
                cmd.Parameters.AddWithValue("@FormulaType",
companyBenefit.benefit.deductionType);
                cmd.Parameters.AddWithValue("@UrlApi",
(object?)companyBenefit.benefit.urlAPI ?? DBNull.Value);
                cmd.Parameters.AddWithValue("@Param1",
companyBenefit.benefit.paramOneAPI);
                cmd.Parameters.AddWithValue("@Param2",
(object?)companyBenefit.benefit.paramTwoAPI ?? DBNull.Value);
                cmd.Parameters.AddWithValue("@Param3",
(object?)companyBenefit.benefit.paramThreeAPI ?? DBNull.Value);
                cmd.Parameters.AddWithValue("@Name",
companyBenefit.benefit.name);
                cmd.Parameters.AddWithValue("@CompanyId",
companyBenefit.benefit.companyId);
                cmd.Parameters.AddWithValue("@MinTime",
companyBenefit.benefit.minEmployeeTime);
                cmd.Parameters.AddWithValue("@Description",
companyBenefit.benefit.description);
                cmd.Parameters.AddWithValue("@ElegibleEmployees",
(object?)companyBenefit.benefit.elegibleEmployees ?? "todos");
            }
        }
    }
}

```

```

        cmd.ExecuteNonQuery();
    }

    transaction.Commit();
    return true;
}
}
catch (Exception ex)
{
    return false;
}
finally
{
    if (_connection.State == ConnectionState.Open)
        _connection.Close();
}
}

```

En el procedimiento almacenado StoreBenefit se emplea una transacción explícita para asegurar la atomicidad y consistencia durante la creación de registros relacionados con beneficios, fórmulas, deducciones y auditoría. Al realizar múltiples inserciones en tablas distintas (Auditoria, Formula, Beneficio, Deduccion) si ocurre algún error, la transacción se revierte completamente mediante un ROLLBACK, evitando así inconsistencias o registros parciales en la base de datos. El manejo de errores está implementado utilizando bloques TRY-CATCH, donde en caso de excepción se asegura la reversión de la transacción y se relanza el error para su adecuada gestión a nivel de aplicación.

Se utiliza el nivel de aislamiento predeterminado de SQL Server, READ COMMITTED, que previene la lectura de datos no confirmados por otras transacciones y garantiza que las validaciones e inserciones se basen en un estado coherente y estable de la base de datos.

10. En editar un beneficio existente de una empresa, en SQLQuery036.sql y CompanyBenefitRepository.cs

```

public void UpdateBenefit(Guid id, CompanyBenefitDTO companyBenefit, string
loggedUserNickname)
{
    try
    {
        if (_connection.State != ConnectionState.Open)

```

```

        _connection.Open();

        using (var transaction = _connection.BeginTransaction())
        {
            companyBenefit.benefit.companyId =
getCompanyId(loggedUserNickname, transaction);
            if (!companyBenefit.benefit.companyId.HasValue)
                throw new Exception("EMPRESA_NO_ENCONTRADA");

            var existingBenefit = getBenefitById(id, transaction);
            if (existingBenefit == null)
                throw new Exception("BENEFICIO_NO_ENCONTRADO");

            if (existingBenefit.benefit.name !=
companyBenefit.benefit.name &&
                dataAlreadyExists("Beneficio", "nombre",
companyBenefit.benefit.name, companyBenefit.benefit.companyId,
transaction))
                throw new Exception("BENEFICIO_DUPLICADO");

            if (existingBenefit.benefit.urlAPI !=
companyBenefit.benefit.urlAPI &&
                (existingBenefit.benefit.paramOneAPI !=
companyBenefit.benefit.paramOneAPI ||
                    existingBenefit.benefit.paramTwoAPI !=
companyBenefit.benefit.paramTwoAPI ||
                    existingBenefit.benefit.paramThreeAPI !=
companyBenefit.benefit.paramThreeAPI))
            {
                if (hasEmployeeWithBenefit(id, transaction))
                    throw new
Exception("BENEFICIO_NO_PUEDE_SER_MODIFICADO");
            }

            using (var cmd = new SqlCommand("UpdateBenefit",
_connection, transaction))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@id", id);
                cmd.Parameters.AddWithValue("@User",
loggedUserNickname);
                cmd.Parameters.AddWithValue("@FormulaType",
companyBenefit.benefit.deductionType);
            }
        }
    }
}

```

```

        cmd.Parameters.AddWithValue("@UrlApi",
(object?)companyBenefit.benefit.urlAPI ?? DBNull.Value);
        cmd.Parameters.AddWithValue("@Param1",
companyBenefit.benefit.paramOneAPI);
        cmd.Parameters.AddWithValue("@Param2",
(object?)companyBenefit.benefit.paramTwoAPI ?? DBNull.Value);
        cmd.Parameters.AddWithValue("@Param3",
(object?)companyBenefit.benefit.paramThreeAPI ?? DBNull.Value);
        cmd.Parameters.AddWithValue("@Name",
companyBenefit.benefit.name);
        cmd.Parameters.AddWithValue("@CompanyId",
companyBenefit.benefit.companyId);
        cmd.Parameters.AddWithValue("@MinTime",
companyBenefit.benefit.minEmployeeTime);
        cmd.Parameters.AddWithValue("@Description",
companyBenefit.benefit.description);
        cmd.Parameters.AddWithValue("@ElegibleEmployees",
(object?)companyBenefit.benefit.elegibleEmployees ?? "todos");

        cmd.ExecuteNonQuery();
    }

    transaction.Commit();
}
}
catch (Exception ex)
{
    throw new Exception("Error al actualizar el beneficio: " +
ex.Message, ex);
}
finally
{
    if (_connection.State == ConnectionState.Open)
        _connection.Close();
}
}
}

```

En el procedimiento almacenado UpdateBenefit se utiliza una transacción explícita para asegurar que todas las actualizaciones relacionadas con un beneficio específico, su fórmula, deducción y auditoría, se realicen de manera atómica y consistente. Al modificar registros en varias tablas vinculadas (Deduccion, Beneficio, Formula, Auditoria), es importante que todas las operaciones tengan éxito para mantener la integridad de los datos; en caso contrario, si alguna actualización falla, la transacción se revierte mediante un ROLLBACK que garantiza que no

queden cambios parciales o inconsistentes. Se usa un bloque TRY-CATCH que maneja adecuadamente las excepciones, asegurando la reversión de la transacción en caso de error y relanzando la excepción para su gestión externa.

El nivel de aislamiento utilizado es el predeterminado en SQL Server, READ COMMITTED, que impide la lectura de datos no confirmados, asegurando que las actualizaciones se basen en un estado consistente y estable de la base de datos. Este nivel de aislamiento protege la integridad referencial y asegura que las modificaciones sobre los beneficios y sus elementos relacionados sean fiables y seguras.

11. En borrar una empresa existente, en SQLQuery038.sql y CompanyDeletionRepository.cs

```
private SqlCommand getDeleteCompanyCommand(string employerEmail)
{
    var storedProcedure = "sp_deleteCompany";
    var command = new SqlCommand(storedProcedure,
        _connectionRepository.connection);
    command.Parameters.AddWithValue("@mailEmployer",
        employerEmail);
    return command;
}
```

El procedimiento almacenado sp_deleteCompany utiliza una transacción explícita con el nivel de aislamiento SERIALIZABLE, el más estricto disponible en SQL Server, para garantizar la máxima integridad durante un proceso crítico: la eliminación (o marcado como borrado) de todos los registros relacionados con una empresa, su empleador y los empleados contratados. Este procedimiento realiza múltiples operaciones complejas sobre una gran cantidad de tablas dependientes, algunas mediante eliminaciones físicas y otras mediante actualizaciones lógicas (campo borrado = 1) si existen planillas asociadas a la empresa. El nivel de aislamiento SERIALIZABLE garantiza que no se permitirán otras transacciones que puedan insertar, actualizar o leer datos relacionados durante el procesamiento de esta eliminación, evitando condiciones de carrera y manteniendo la coherencia referencial. La transacción se encapsula dentro de un bloque TRY-CATCH, permitiendo que, ante cualquier error inesperado, se realice un ROLLBACK que revierte todos los cambios y se propaga la excepción con detalles adecuados. El uso de variables tipo tabla (@idEmployees, @idBenefits, etc.) permite estructurar y controlar las eliminaciones de forma eficiente, asegurando que los registros dependientes se eliminen o marquen de forma ordenada según las reglas del negocio.

12. Borrar un beneficio existente de una empresa, en SQLQuery042.sql y CompanyBenefitRepository.cs

```

public void DeleteBenefit(Guid id, string loggedUserNickname)
{
    try
    {
        if (_connection.State != ConnectionState.Open)
            _connection.Open();

        using (var transaction = _connection.BeginTransaction())
        {
            using (var cmd = new SqlCommand("DeleteBenefit",
loggedUserNickname);
            _connection, transaction))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@id", id);
                cmd.Parameters.AddWithValue("@User",
loggedUserNickname);
                cmd.ExecuteNonQuery();
            }
            transaction.Commit();
        }
    }
    catch (Exception ex)
    {
        throw new Exception("Error al eliminar el beneficio: " +
ex.Message, ex);
    }
    finally
    {
        if (_connection.State == ConnectionState.Open)
            _connection.Close();
    }
}

```

El procedimiento almacenado DeleteBenefit emplea una transacción explícita para garantizar la integridad y consistencia de los datos durante el proceso de eliminación de un beneficio. Dentro de la transacción, primero se valida si el beneficio ya ha sido marcado como borrado, en cuyo caso no se realiza ninguna acción adicional. Si no existen registros asociados en BeneficioPorEmpleado ni en DeduccionAPago, se realiza un borrado físico del beneficio y sus deducciones. En caso contrario, se procede con un borrado lógico, actualizando los campos borrado de las tablas Beneficio, Deduccion, Formula, y BeneficioPorEmpleado, y se actualiza también la auditoría con el último usuario modificador. Además, si el beneficio aún no ha sido reflejado en ninguna planilla, se desactiva su elegibilidad para futuros pagos. El uso de la transacción asegura que estas operaciones múltiples se ejecuten como una unidad indivisible,

evitando inconsistencias en caso de error. El manejo de errores se realiza mediante un bloque TRY-CATCH, el cual revierte la transacción si ocurre una excepción, preservando así la integridad del sistema.

Se utiliza el nivel de aislamiento predeterminado (READ COMMITTED), suficiente en este contexto para evitar lecturas sucias y mantener un estado de datos confiable durante la ejecución del procedimiento.